
# Advanced Programming

## Exceptions

# What Is an Exception?

An "abnormal event" that occurs during the execution

```
public class Example {
  public static void main(String args[]) {
    int v[] = new int[10];
    v[10] = 0; //Oops... !
    System.out.println("Hello world ...?!");
  }
}
```

**"Exception in thread "main"**
**java.lang.ArrayIndexOutOfBoundsException :10**
**at Example.main (Example.java:4)"**

*"throw an exception"*

*"catch the exception"*

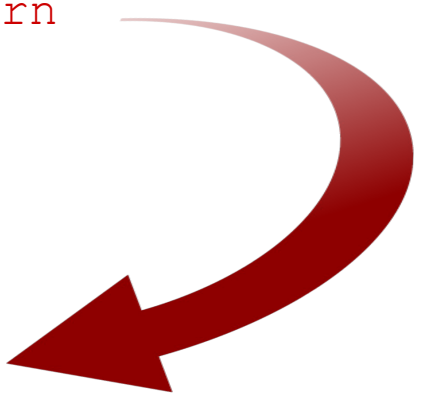
*"exception handler"*

# Catching and Handling Exceptions

## *try - catch - finally*

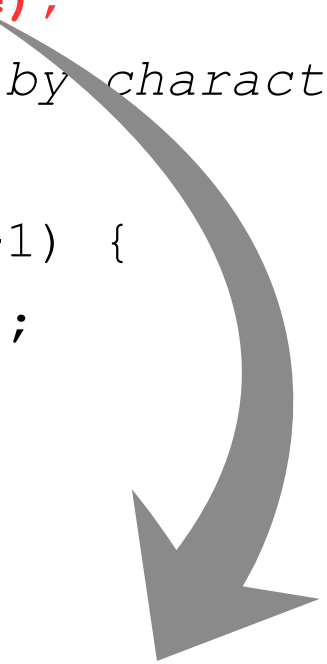
```
try {
  // Block of instructions
  methodX()
  methodY()
  methodZ()
}
catch (ExceptionType1 variable) {
  // Handling exceptions of type 1
}
catch (ExceptionType2 variable) {
  // Handling exceptions of type 2
}
catch (ExceptionType3 | ExceptionType4 variable) {
  // Handling exceptions of type 3 or 4
}
finally {
  // Cleanup code: executes whether or not an exception is thrown
}
...execution continues
```

A new exception is born



# Example – Reading a File

```
public static void readFile(String filename) {
   FileReader f = null;
   // Open the file
   f = new FileReader(filename);
   // Read the file character by character
   int c;
   while ( (c = f.read()) != -1) {
      System.out.print((char)c);
   }
   // Close the file
   f.close();
}
```



**unreported exception FileNotFoundException;**
**must be caught or declared to be thrown**

# “Catching” I/O Exceptions

```
public static void readFile(String filename) {
  FileReader f = null;
  try {
    f = new FileReader(filename);    // Open the file
    int c;                           // Read the file
    while ( (c=f.read()) != -1) {
      System.out.print((char)c);
    }
  } catch (FileNotFoundException e) {
    System.err.println("The file " + filename + "is missing!");
  } catch (IOException e) {
    System.out.println("Unexpected error reading the file!");
    e.printStackTrace();
  } finally {
    if (f != null) {                 // Close the file
      try {
        f.close();
      } catch (IOException e) {
        System.err.println("Error closing the file!");
      }
    }
  }
}
```

# “Throwing” exceptions

```
[modifiers] ReturnedType method([arguments])
   throws ExceptionType1, ExceptionType2, … { }
```

```
public class FileReadExample {
  public static void readFile(String filename)
      throws FileNotFoundException, IOException {
    FileReader f = new FileReader(filename);
    int c;
    while ( (c = f.read()) != -1)
      System.out.print((char)c);
    f.close();
  }
  public static void main(String args[]) {
    try {
      readFile(args[0]);
    } catch (FileNotFoundException e) {
      System.err.println("File not found...");
    } catch (IOException e) {
      System.out.println("I/O Error");
    } catch (Exception e){
      System.out.println("Something is wrong..." + e);
    }
  }
}
```

# *try - finally*

```
public static void readFile(String filename)
    throws FileNotFoundException, IOException {
  FileReader f = null;
  try {
    f = new FileReader(filename);
    int c;
    while ( (c=f.read()) != -1)
      System.out.print((char)c);
  }
  finally {
    if (f!=null)
      f.close();
  }
}
```

# *try-with-resources*

*Resource* - objects that must be closed after using them; they are of type *AutoCloseable*

```
try(FileReader f = new FileReader(filename)) {
  int c;
  while ( (c = f.read()) != -1)
    System.out.print((char)c);
}
```

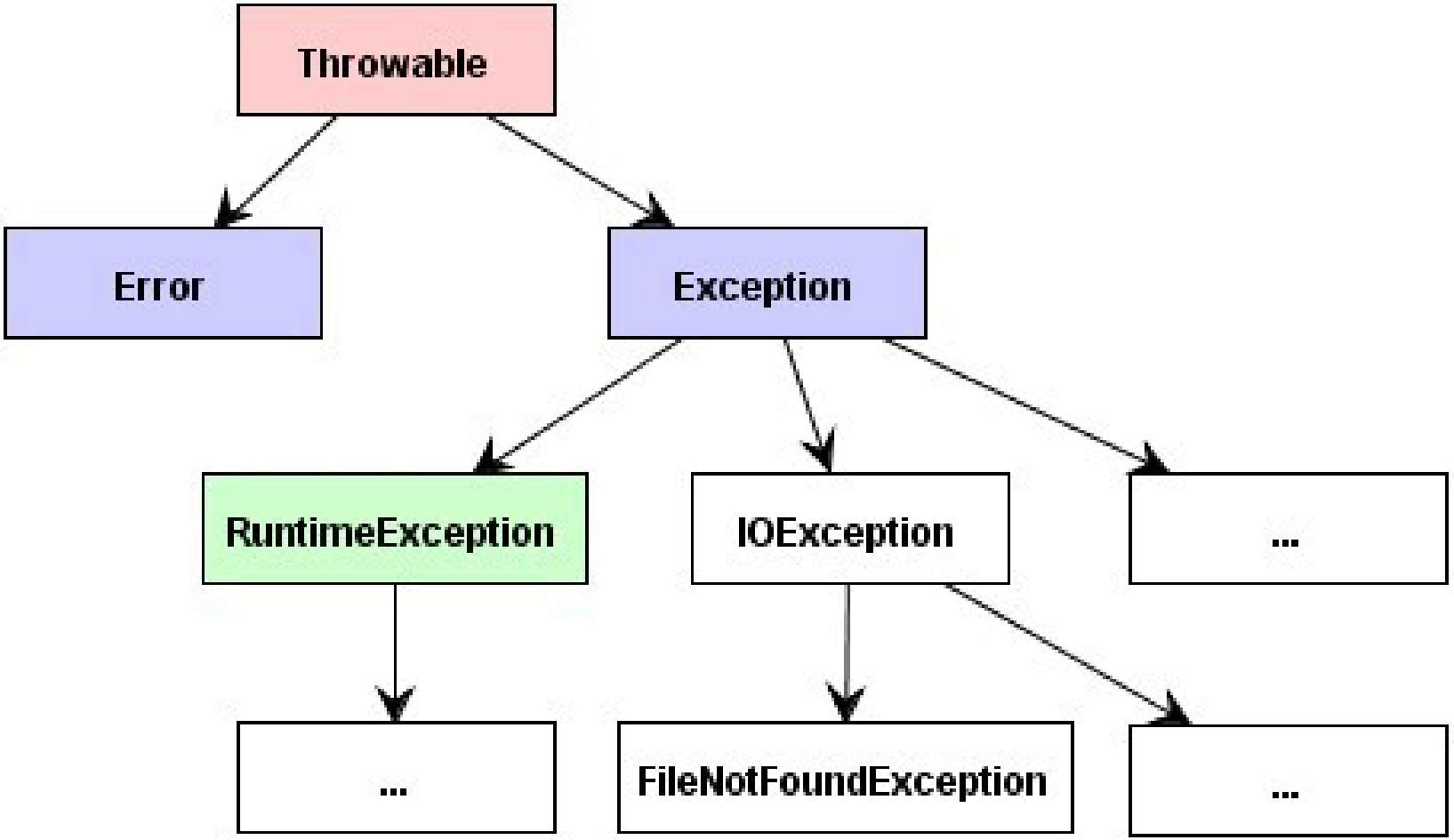
The exception thrown when the *FileReader* was closed is suppressed

```
try (Connection con = createConnection();
     Statement stmt = con.createStatement();
     ResultSet rs = stmt.executeQuery(query)) {

  return (rs.next() ? rs.getObject(1) : null);
} catch (SQLException e) {
  System.err.println(e);
}
```

The resources will be closed in reverse order

The **close()** method of an **AutoCloseable** object is called automatically when exiting a try-with-resources block for which the object has been declared in the resource specification header.

# Exceptions Class Hierarchy



# *Checked* versus *Unchecked*

- *Checked Exceptions*
  - Abnormal situations that can not be controlled at the time of writing the code (design-time): *file system* errors, *network* communications errors, etc.
  - Must be handled ("caught" or "thrown")
  - Extend *Exception*: `IOException, SQLException, etc.`
- *Unchecked Exceptions*
  - Errors caused by situations out of which the application can not recover, usually *programming mistakes*.
  - Do not need to be handled (but it is possible)
  - Extend either *Error* or *RuntimeException*:
    `NullPointerEception, IllegalArgumentException, ArithmeticException, ArrayIndexOutOfBoundsException, etc.`

# *Error*

- Indicates a **serious problem** that a reasonable application should not try to catch. Most such errors are abnormal conditions.
- *Unchecked*
- Examples:
  - **`VirtualMachineError`**

    (Thrown to indicate that the Java Virtual Machine is broken or has run out of resources necessary for it to continue operating.)

    - **`InternalError, UnknownError,`**
    - **`OutOfMemoryError, StackOverflowError`**

# Who Creates the Exceptions?

## The throw Statement

- The author of a method will signal exceptional situations **creating** and throwing exceptions.

```
public class Stack {
  …                          //throws is mandatory for checked exceptions
  public Object peek() throws EmptyStackException {
    int len = size();
    if (len == 0) throw new EmptyStackException();
    return elementAt(len - 1);
  }
}
```

- The Virtual Machine, for "standard" *RuntimeExceptions*

# Validating Method Arguments

```
class Person {
  private String name;
  private int age;

  public void setName(String name) {
    if (name == null || name.trim().equals("")) {
        throw new IllegalArgumentException(
          "Name should not be empty.");
    }
    if (!name.matches("[a-zA-Z]+")) {
        throw new IllegalArgumentException(
          "Name should only contain characters: " + name);
    }
    this.name = name;
  }

  public void setAge(int age) {
    if (age < 0) {
      throw new IllegalArgumentException(
        "Age should be a positive number: " + age);
    }
    this.age = age;
  }
  ...
}
```

# Reusing Validation Rules

- Verifying data integrity is a **repetitve work**
- Error messages may also **vary by locale**
- Consider creating your own "**validation framework**" or use an existing one, such as Apache Commons Validator (or other)
- Example: implement a class for validating emails:

```
public void setEmail(String email) {
  new EmailValidationRule().validate(email);
  ...
}
```

# Creating Custom Exceptions

- Extend a proper subclass of *Throwable*
- *Checked* vs. *Unchecked*:

  *If a client can reasonably be expected to recover from an exception, make it a checked exception. If a client cannot do anything to recover from the exception, make it an unchecked exception.*

```
public class MyOwnException extends Exception {

  //Usually, define custom properties and constructors

  public MyOwnException(String message) {
    super(message);
  }
}
```

**RuntimeException**

# Example

- Define your custom exception

```
public class InvalidAgeExeption extends RuntimeException {
    public InvalidAgeExeption(String message) {
        super(message);
    }
    public InvalidAgeExeption(int age) {
        super("Invalid age: " + age);
    }
}
```

- Use your custom exception

```
public void setAge(int age) {
  if (age < 0)  throw new InvalidAgeException(age);
  if (age > 18) throw new InvalidAgeException("Sorry, too old");
  this.age = age;
}
```

# Exception Chaining (Wrapping)

```
try {
   Person person =
     database.readPerson(personId);

} catch (SQLException sqlException) {
    // catch the original exception
    System.err.println(sqlException);

    // create a new, custom exception
    // wrapping the original exception
    MyException myException =
      new MyException("Database Error", sqlException);
    myException.setDetail("Invalid person id " + personId);

    // throw the custom exception
    throw myException;
}
```

# Code Separation

"Spaghetti" Code (tangled, unstructured)

```
int readFile() {
  int codEroare = 0;
  open the file;
  if (the file has opened) {
    determine its size;
    if (the size was determined ) {
      allocate memory;
      if (the memory was allocated) {
        read the file into memory;
        if (cannot read from file){
          errorCode = -1;
        }
      } else {
        errorCode = -2;
      }
    }
    ...
  }}}}}}}}}}}}}}}}}}}}}}}}}}}}}}}}}}}}
  return errorCode;
}
```

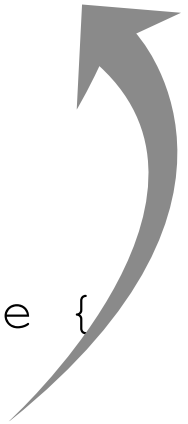
```
int readFile() {
try {
  open the file;
  determine its size;
  allocate memory;
  read the file into memory;
  close the file;

} catch(file didn't open){
  handle this exception
} catch (cannot determine size){
  handle this exception
} catch (not enough memory) {
  handle this exception
} catch (file read failed) {
  handle this exception
} catch (cannot close the file) {
  handle this exception
}
```

# Grouping and Differentiating Error Types

```
try {

  String driverName = new String(
    Files.readAllBytes(Paths.get("driver.txt")));

  Class.forName(driverName).newInstance();

} catch (IOException ex) {
  // problems with the file

} catch (ClassNotFoundException ex) {
  // problem with the driver class

} catch (IllegalAccessException ex) {
  // cannot acces the class

} catch (InstantiationException ex) {
  // cannot instantiate the class
}
```

# ✔ Propagating Errors

```
int method1() {
  try {
    method2();
  } catch (ExceptionType e) {
    //handle the exception
  }
}
int method2() throws ExceptionType {
  ...
  method3();
  ...
}

int method3() throws ExceptionType {
  ...
  throw new ExceptionType();
  ...
}
```




# Advanced Programming

## Input / Output Streams

## File I/O

# The Context

- How do we **read from a file**, or **write to a file**:
  - *text:* characters or lines of text, ...?
  - *binary:* arrays of bytes ...?
- How do we **send or receive**:
  - objects in a distributed application?
  - data to/from a serial port?
- How do threads communicate asynchronous?
- How do we write very large sets of data into a relational database?
- ...

# I/O Streams

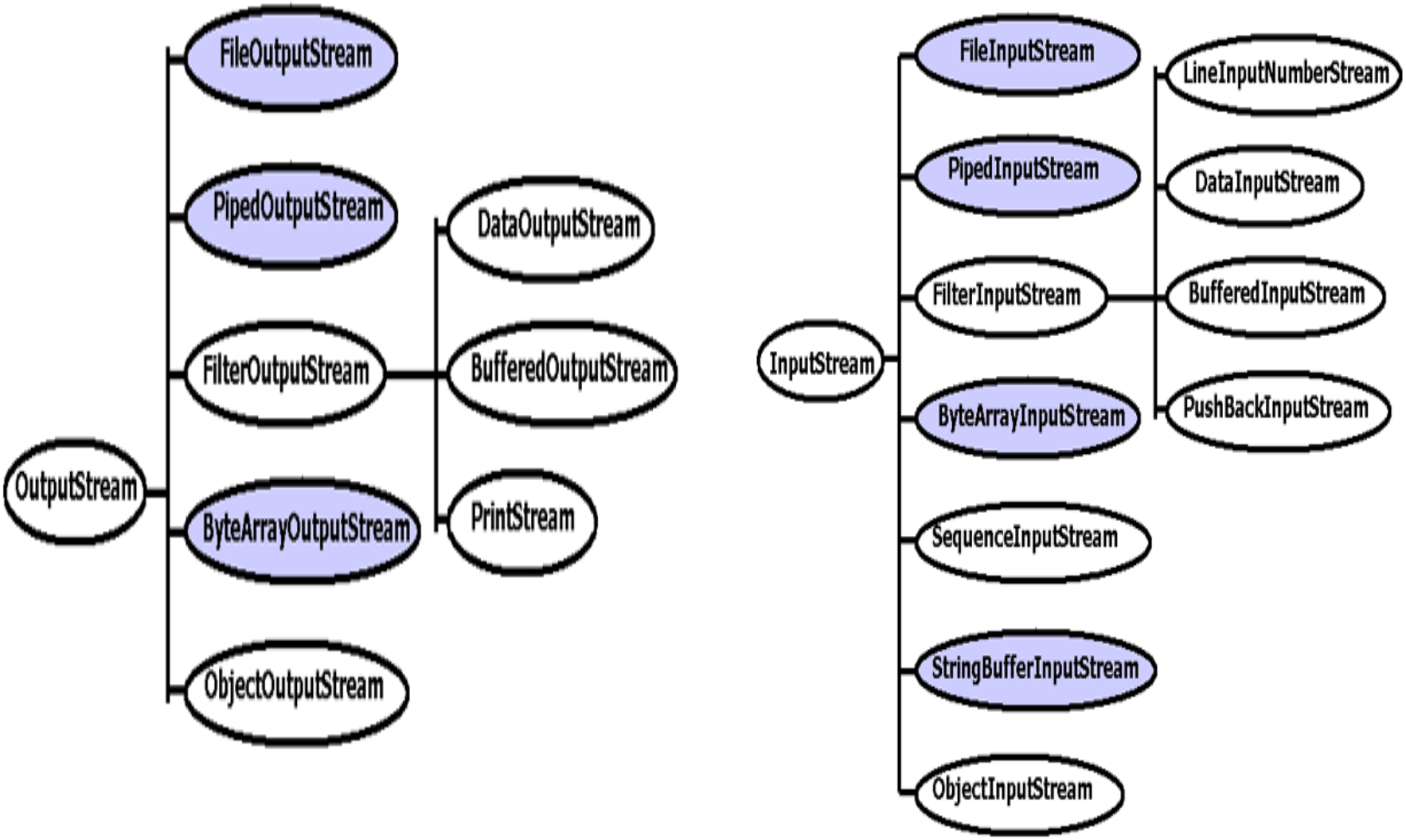
- A stream is a sequence of data: serial, unidirectional, having a source / destination.
- Byte Streams (8 bits) / Char Streams(16 bits)
- *Producers (Output Streams)*


- *Consumers (Input Streams)*

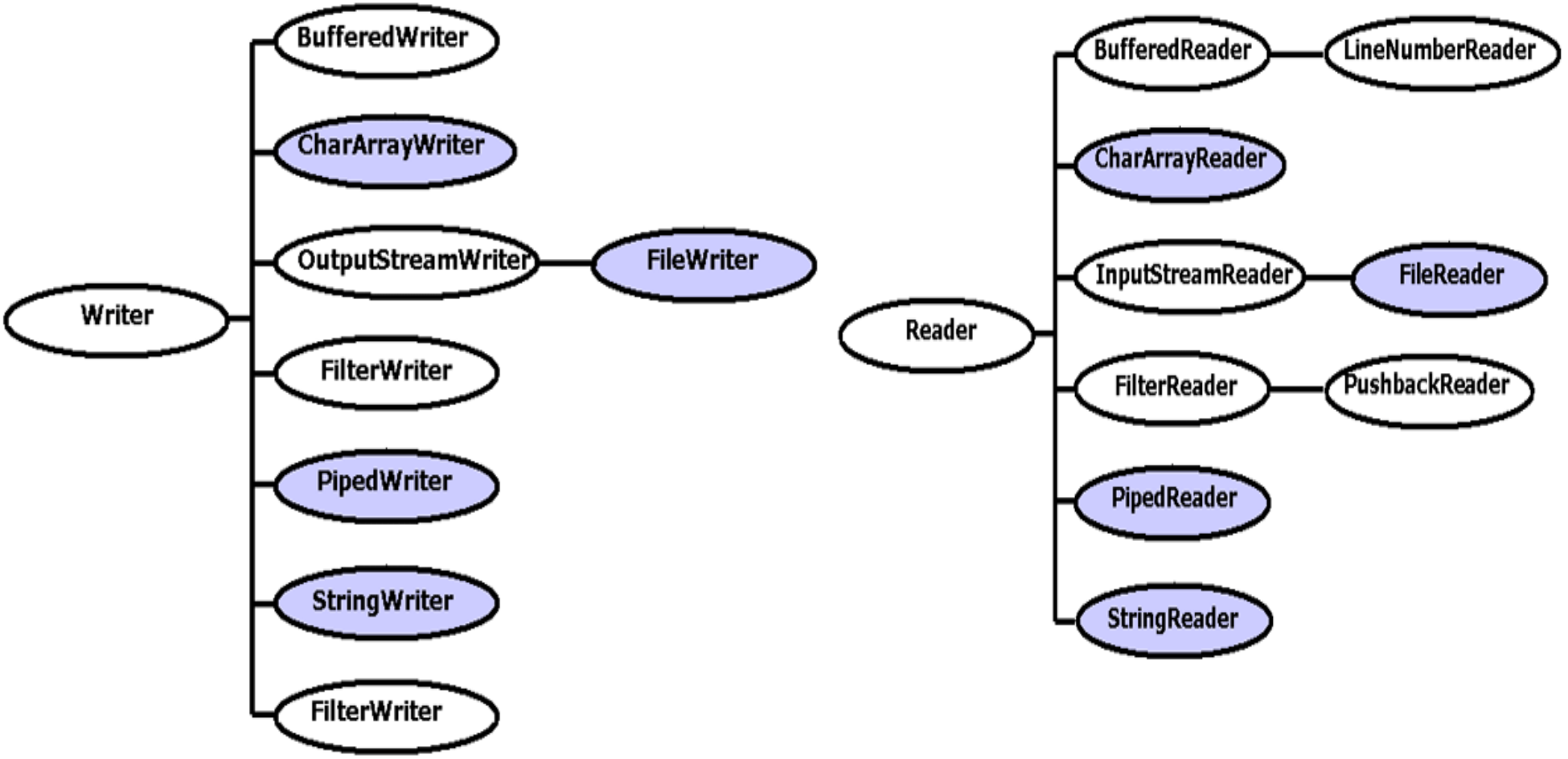
# Using I/O Streams

```
import java.io.*;

open the stream; //new StreamClass([arguments]);

while (there is more data to read or write) {

  read/write some data;

  //invoke read(), write() or other specialized methods

}

close the stream; //...dont't forget to invoke close()

take care of the possible exceptions: IOExeption;
```

# Byte Streams



# Character Streams



# Primitive Streams vs. Decorators

- *Primitive* streams “know” to effectively communicate (read or write) with an external “partener” (file, memory, thread, etc.). Examples:
  - `FileReader, FileWriter`
  - `ByteArrayInputStream,ByteArrayOutputStream`
  - `PipedReader, PipedWriter`
- *Decorator* streams “know” to communicate with another stream (primitive or not) in order to process the raw data and offer specialized methods:
  - `BufferedReader, BufferedWriter, PrintWriter`
  - `DataInputStream, DataOutputStream`
  - `ObjectInputStream, ObjectOutputStream`

# Creating Streams

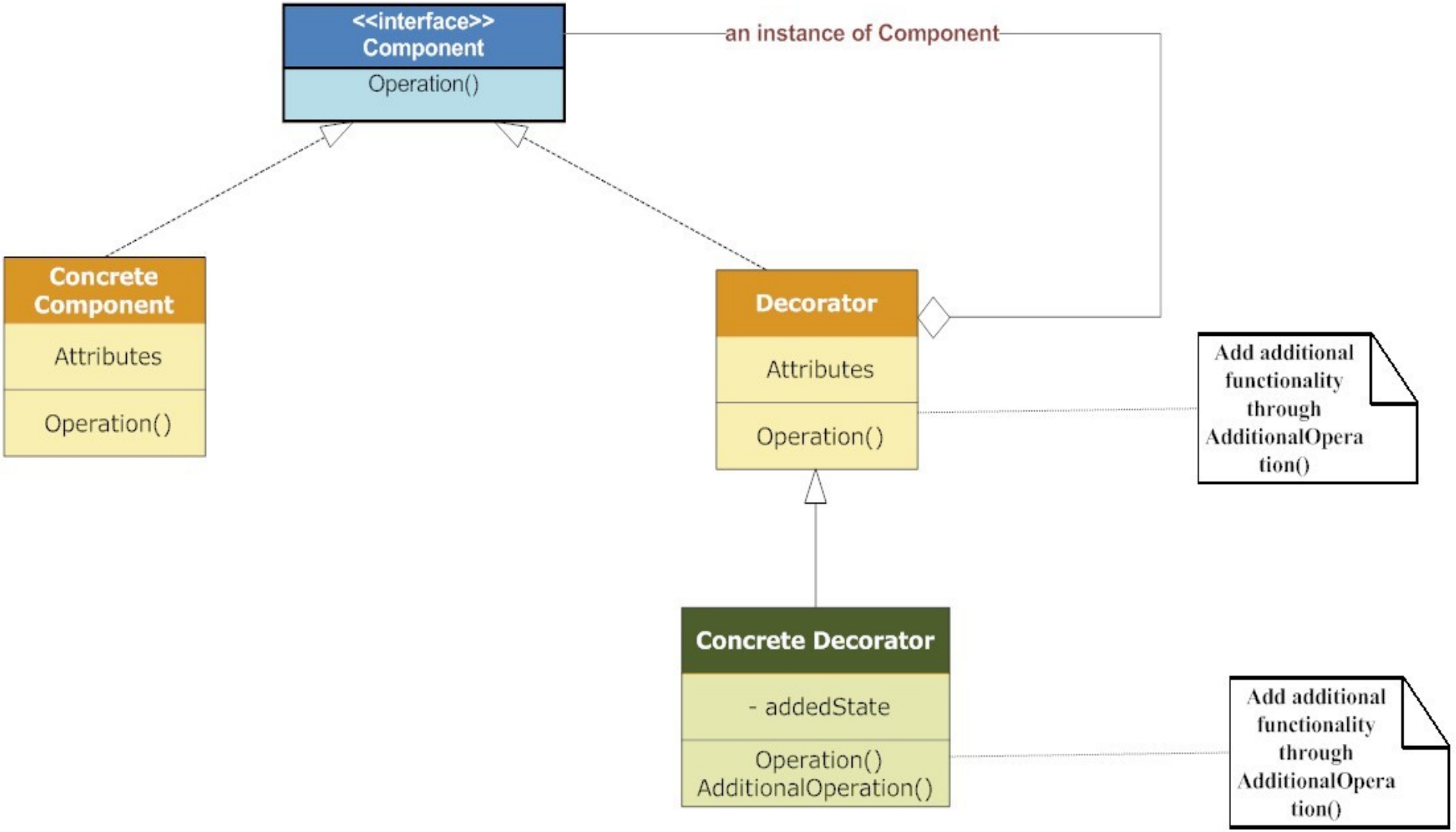
```
PrimitiveStream stream =
        new PrimitiveStream(externalResource);
DecoratorStream decorator =
        new DecoratorStream(stream);
```

**Exemplu:**

```
FileReader fileReader = new FileReader("fisier.txt");
BufferedReader bufferReader =
    new BufferedReader(fileReader);
String line = bufferedReader.readLine();
```

# Decorator Design Pattern

Decorator Pattern Structure



# Decorator: *Reader*

- Abstract class for reading character streams. The only methods that a subclass must implement are read(char[], int, int) and close(). Most subclasses, however, will override some of the methods defined here in order to provide higher efficiency, additional functionality, or both.

```
public abstract class Reader implements Readable, Closeable {

    …

    public abstract int read(char cbuf[], int off, int len)
    throws IOException;

    public abstract void close() throws IOException;

    …

}
```

# Decorator: *BufferedReader*

- Reads text from a character-input stream, buffering characters so as to provide for the efficient reading of characters, arrays, and lines..

```
public class BufferedReader extends Reader { {
   private Reader in; //the decorated object
   public BufferedReader(Reader in) {
      this.in = in;
      …
   }
   public String readLine() throws IOException {
      …
      in.read( … );
      …
   }
}
```

# Buffered Streams

**`BufferedReader, BufferedWriter, BufferedInputStream, BufferedOutputStream`**

Read /Write data from a stream, buffering elements so as to provide for the efficient reading of arrays, and lines.

```
BufferedOutputStream out = new BufferedOutputStream(
  new FileOutputStream("out.dat"), 1024)
  //1024 is the size of the buffer

for(int i=0; i<100; i++) {
  out.write(i);
  //the buffer is not full yet, the file contains nothing
}

out.flush();
//the buffer is flushed, data is written to the file
```

- Greatly reduces the access to the external resource
- Increases the execution speed

# Serializing Primitive Data

**DataInputStream, DataOutputStream**

- Writing/Reading primtive data in binary format, "*in a machine-independent way*"

```
//Writing
FileOutputStream fos = new FileOutputStream("test.dat");
DataOutputStream out = new DataOutputStream(fos);
out.writeInt(12345);
out.writeDouble(12.345);
out.writeBoolean(true);
out.writeUTF("Sir de caractere");
out.flush();
fos.close();
...
//Reading
FileInputStream fis = new FileInputStream("test.dat");
DataInputStream in  = new DataInputStream(fis);
int i = in.readInt();
double d = in.readDouble();
boolean b = in.readBoolean();
String s = in.readUTF();
fis.close();
```

# Serialiazing Objects

**ObjectInputStream, ObjectOutputStream**

- Writing/Reading objects in a binery format, "*in a machine-independent way*"

```
//Writing
FileOutputStream fos = new FileOutputStream("test.ser");
ObjectOutputStream out = new ObjectOutputStream(fos);
out.writeObject(new Date());
out.writeObject("Hello World");
out.writeInt(12345);
out.flush();
fos.close();

//Reading
FileInputStream fis = new FileInputStream("test.ser");
ObjectInputStream in = new ObjectInputStream(fis);
Date date = (Date)in.readObject();
String message = (String)in.readObject();
int i = in.readInt();
fis.close();
```

# Standard I/O Streams

- **`System.in` - `InputStream`**
- **`System.out` - `PrintStream`**
- **`System.err` - `PrintStream`**

Redirecting the standard streams:

`setIn(InputStream)` - redirecting the input

`setOut(PrintStream)` - redirecting the output

`setErr(PrintStream)` - redirecting the error stream

Example:

```
PrintStream fis = new PrintStream(new FileOutputStream("results.txt")));
System.setOut(fis);
PrintStream fis = new PrintStream(new FileOutputStream("errors.txt")));
System.setErr(fis);
```

# Scanning and Formatting

**java.util.Scanner,**

**java.util.Formatter, java.text.Format**

```
Scanner s = Scanner.create(System.in);
String name = s.next();
int age = s.nextInt();
double salary = s.nextDouble();
s.close();
```

```
System.out.printf("%s %8.2f %n",name, salary, age);
SimpleDateFormat dateFormat =
  new SimpleDateFormat("dd-MM-yyyy");
String date = dateFormat.format(today);
System.out.println("Today in dd-MM-yyyy format : " + date);
```

# “Useful” I/O Classes

- *java.io.File*

  An abstract representation of file and directory pathnames.

- *java.io.RandomAccesFile*

  Supports both reading and writing to a file, using a file pointer.

- *java.io.StreamTokenizer*

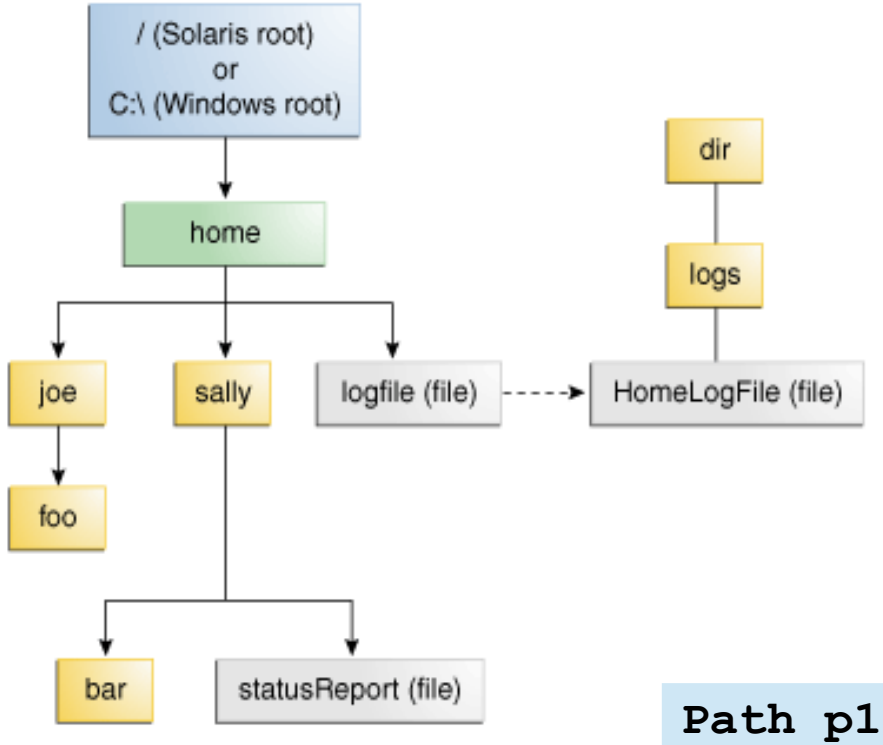
  Takes an input stream and parses it into "tokens", allowing the tokens to be read one at a time.

  (see also *StringTokenizer, String.split*)

- *… here comes the New Java I/O → java.nio*

# File I/O (Featuring NIO)

- *java.io.File* has some "issues" (some methods didn't throw exceptions when they failed, no support fo symbolic links, no consistency across platforms, not scalable, no support for recursively walk a file tree, etc.)
- *java.nio.file.Path* is the modern replacement of *File.*




A *Path* object is a programmatic representation of **a path in the file system.** May be:

- relative
- absolute
- symbolic (soft) link
- file
- directory (folder)
- might not exist

```
Path p1 = Paths.get("/home/joe/foo");
```

# File Operations

- *java.nio.file.Files* - Static methods that operate on files, directories, or other types of files.
- Examples:

```
boolean readable = Files.isReadable(file);
Files.copy(source, target, REPLACE_EXISTING);
Files.move(source, target, ATOMIC_MOVE);
Files.delete(path);
Path newFile = Files.createFile(path);
Path newDir = Files.createDirectory(path);

BasicFileAttributes attr =
 Files.readAttributes(file, BasicFileAttributes.class);
System.out.println("lastAccessTime: " + attr.lastAccessTime());
...
```

# Traversing a Directory Tree

*Files.**walkFileTree**(path, new **FileVisitor**<Path>() { … });*

```
Files.walkFileTree(path, new FileVisitor<Path>() {
  public FileVisitResult preVisitDirectory(Path dir, BasicFileAttributes
attrs) throws IOException {
    System.out.println("pre visit dir:" + dir);
    return FileVisitResult.CONTINUE;
    // or TERMINATE or SKIP_SIBLINGS or SKIP_SUBTREE
  }
  public FileVisitResult visitFile(Path file, BasicFileAttributes attrs)
throws IOException {
    System.out.println("visit file: " + file);
    return FileVisitResult.CONTINUE;
  }
  public FileVisitResult visitFileFailed(Path file, IOException exc)
throws IOException {
    System.out.println("visit file failed: " + file);
    return FileVisitResult.CONTINUE;
  }
  public FileVisitResult postVisitDirectory(Path dir, IOException exc)
throws IOException {
    System.out.println("post visit directory: " + dir);
    return FileVisitResult.CONTINUE;
  }
});
```

# Reading and Writing

- Commonly Used Methods for Small Files

```
byte[] fileArray = Files.readAllBytes(path);
List<String> fileLines = Files.readAllLines(path);
String fileContent = new String(Files.readAllBytes(path));
Files.write(path, buffer); // to write bytes, or lines
```

- Buffered I/O Methods for Text Files

```
BufferedReader reader = Files.newBufferedReader(path);
BufferedWriter writer = Files.newBufferedWriter(path);
```

- Unbuffered Streams

```
InputStream in = Files.newInputStream(path);
OutputStream out = Files.newOutputStream(path);
```

- Methods for *Channels* and *ByteBuffers*

*While stream I/O reads a character at a time, channel I/O reads a buffer at a time*

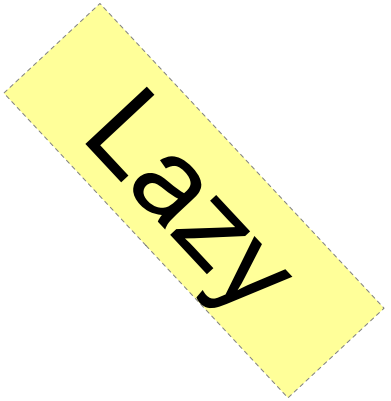
# Using Streams and Files

- Iterating over the lines of a text file

```
Path path = Paths.get("c:\\data\\SomeFile.txt");
try (Stream<String> lines = Files.lines(path)) {
   lines.forEachOrdered(line->System.out.println(line));
} catch (IOException e) {
   System.err.println(e);
}
```

- Iterating over the files in a directory

```
Path path = Paths.get("c:\\data");
Files.list(dir)
   .filter((Path file) ->
      file.getFileName().toString().endsWith(".pdf"))
   .forEach(System.out::println);
```



# Watching a Directory for Changes

- Watch Service API
- Create and Register

```
WatchService watcher = FileSystems.getDefault().newWatchService();
Path dir = Paths.get("Path/To/Watched/Directory");
dir.register(watcher, ENTRY_CREATE, ENTRY_DELETE, ENTRY_MODIFY);
```

- Watch...

```
while (true) {
    WatchKey key = watcher.take(); // wait for a key to be available
    for (WatchEvent<?> event : key.pollEvents()) {
        WatchEvent.Kind<?> kind = event.kind();
        WatchEvent<Path> ev = (WatchEvent<Path>) event;
        Path fileName = ev.context();

        System.out.println(kind.name() + ": " + fileName);
       key.reset();
    }
}
```